Vulnerability Assessment Report: Support Ticket System

Author: Weilong Xu (Edison)

1. Executive Summary

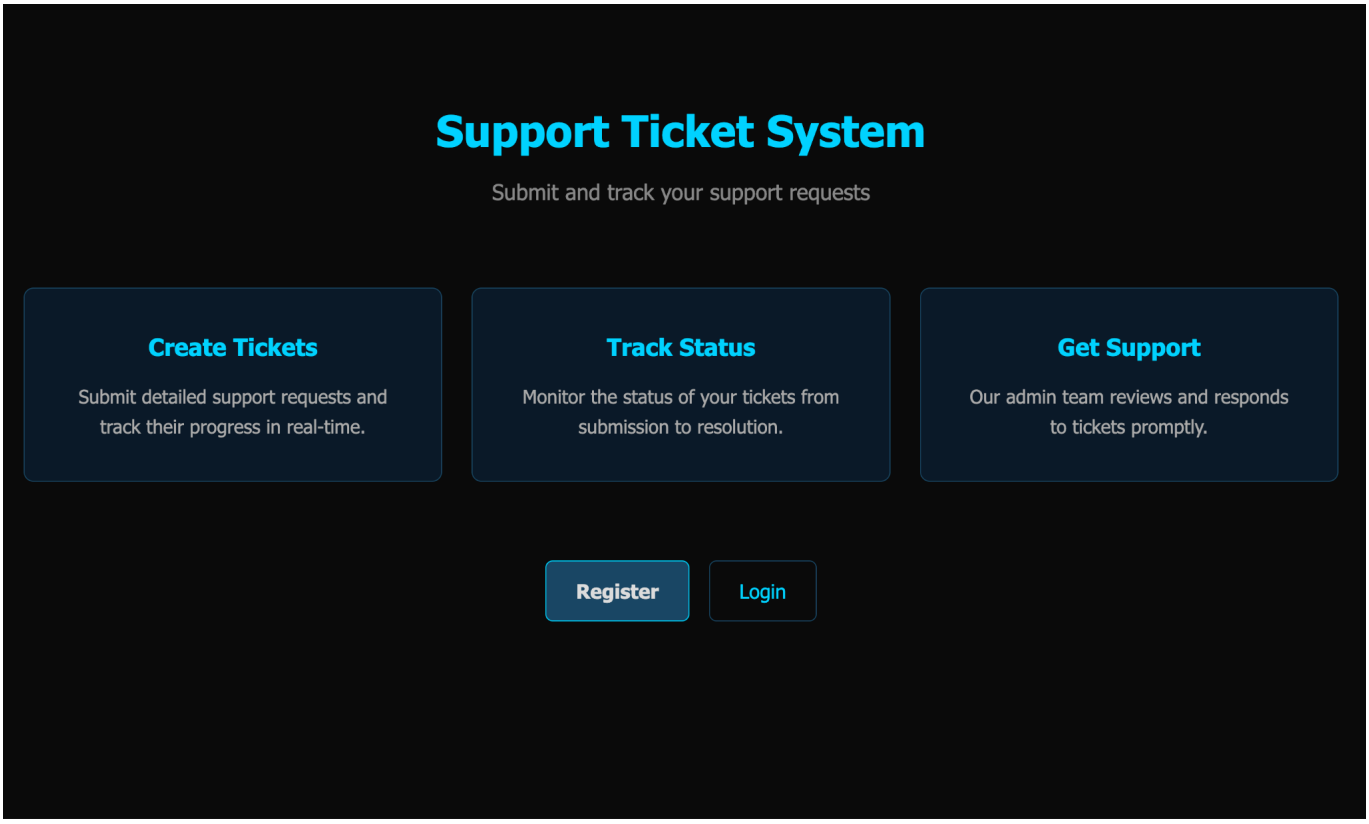
During the security assessment of the Support Ticket System, I found a critical Stored Cross-Site Scripting (XSS) vulnerability that allowed me to completely take over an administrative session and extract sensitive data. First, I discovered an automated administrative review function that interacts with user-submitted links. Then, I found that the ticket creation form blindly trusts user input without sanitizing it. By combining these two flaws, I forced an administrator bot to execute exploits. This allowed me to grab the restricted /admin/dashboard data and steal the hidden system flag, sending it directly to my own external server.

2. Technical Findings

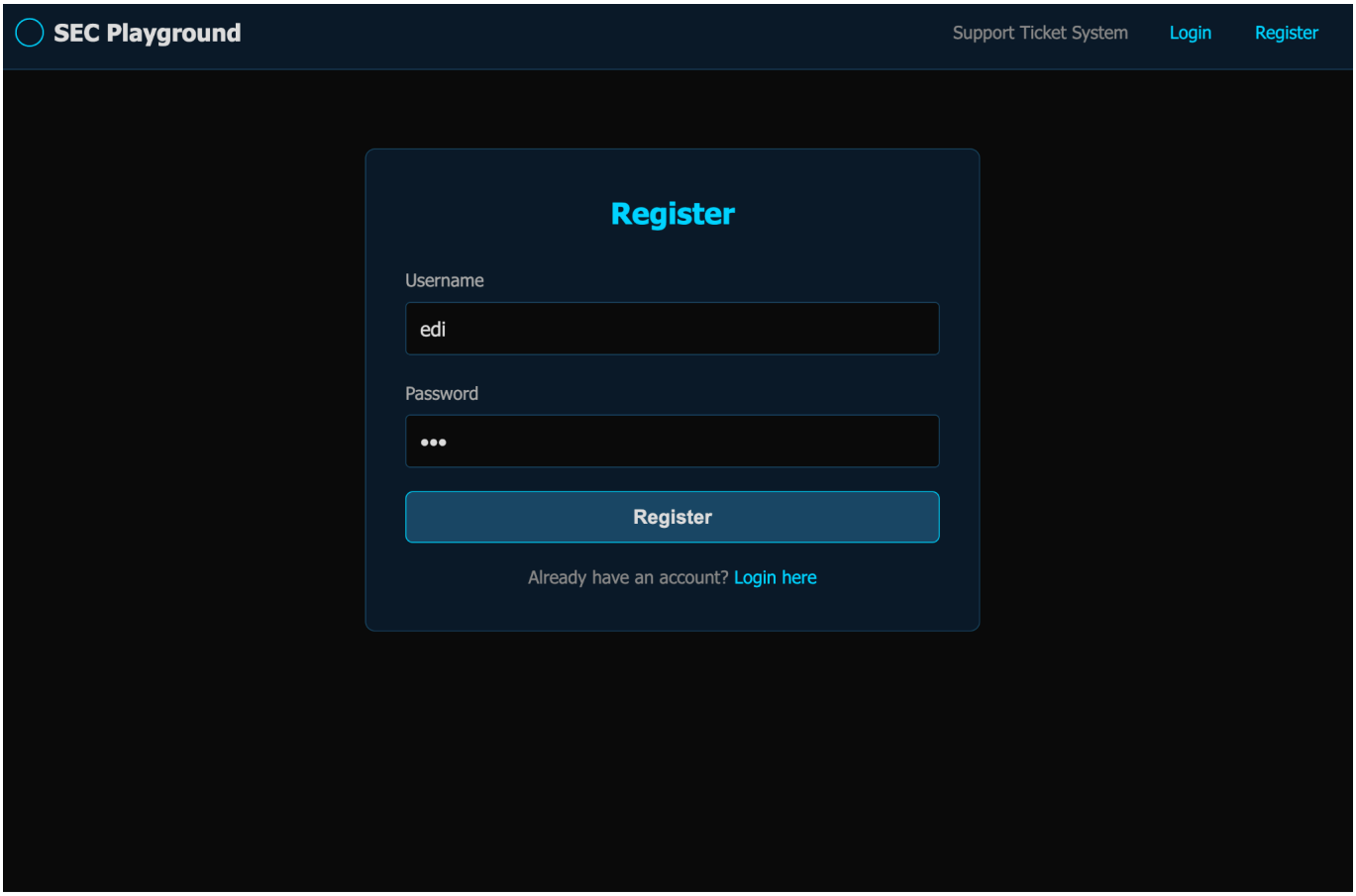
Here is the step-by-step breakdown of how I discovered and exploited the vulnerability to extract the flag.

2.1. Exploring & Testing

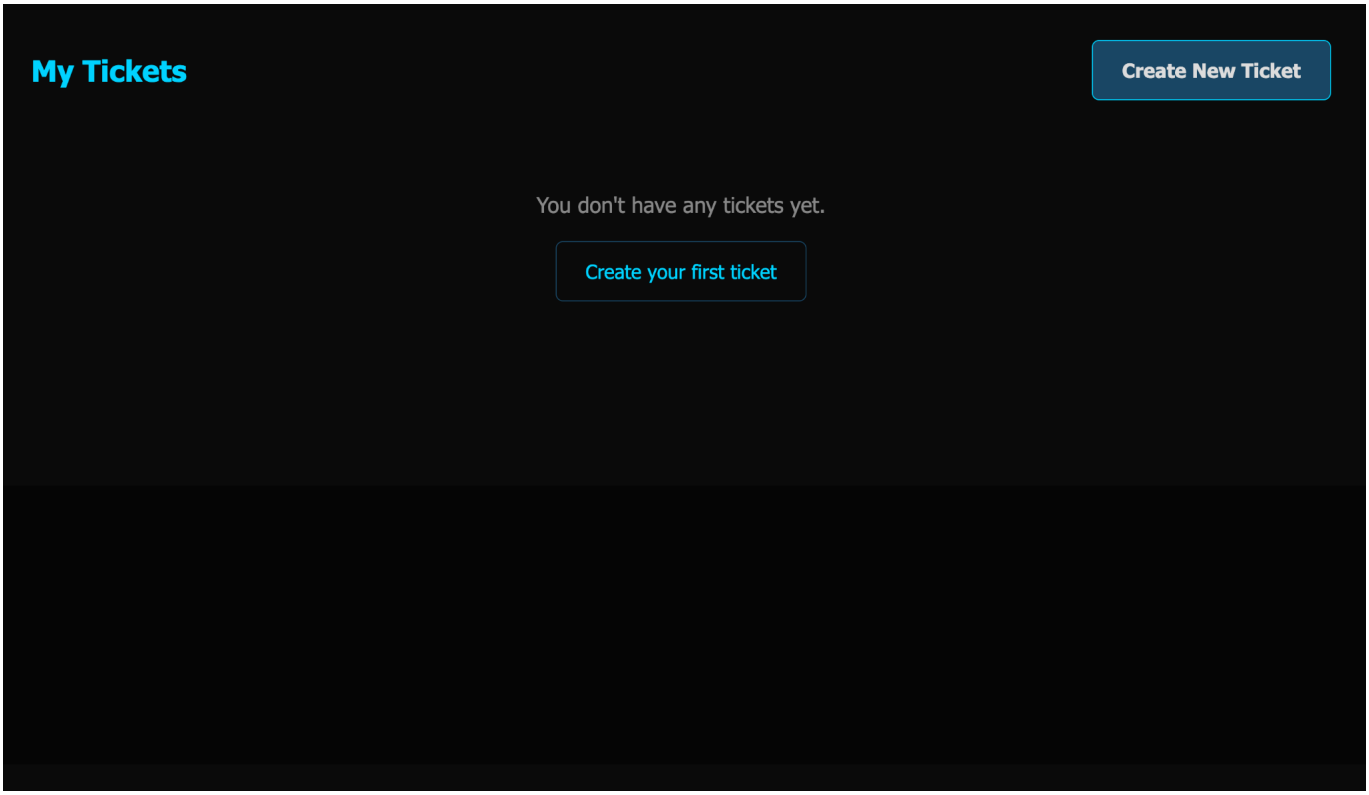
I started by reviewing the initial landing page of the Support Ticket System to understand its basic functionality.



To look at the internal features, I registered a standard user account with the username edi.

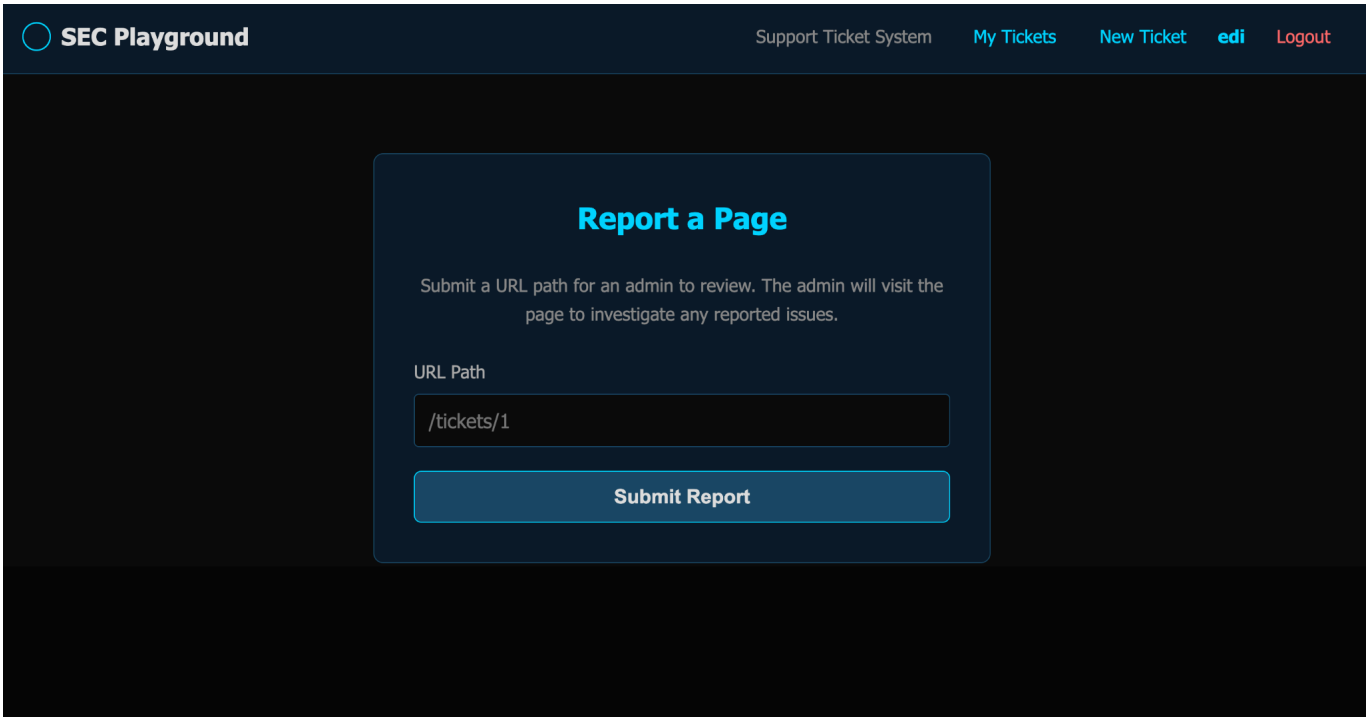


Upon logging in, It shows an empty dashboard showing I had no active tickets.

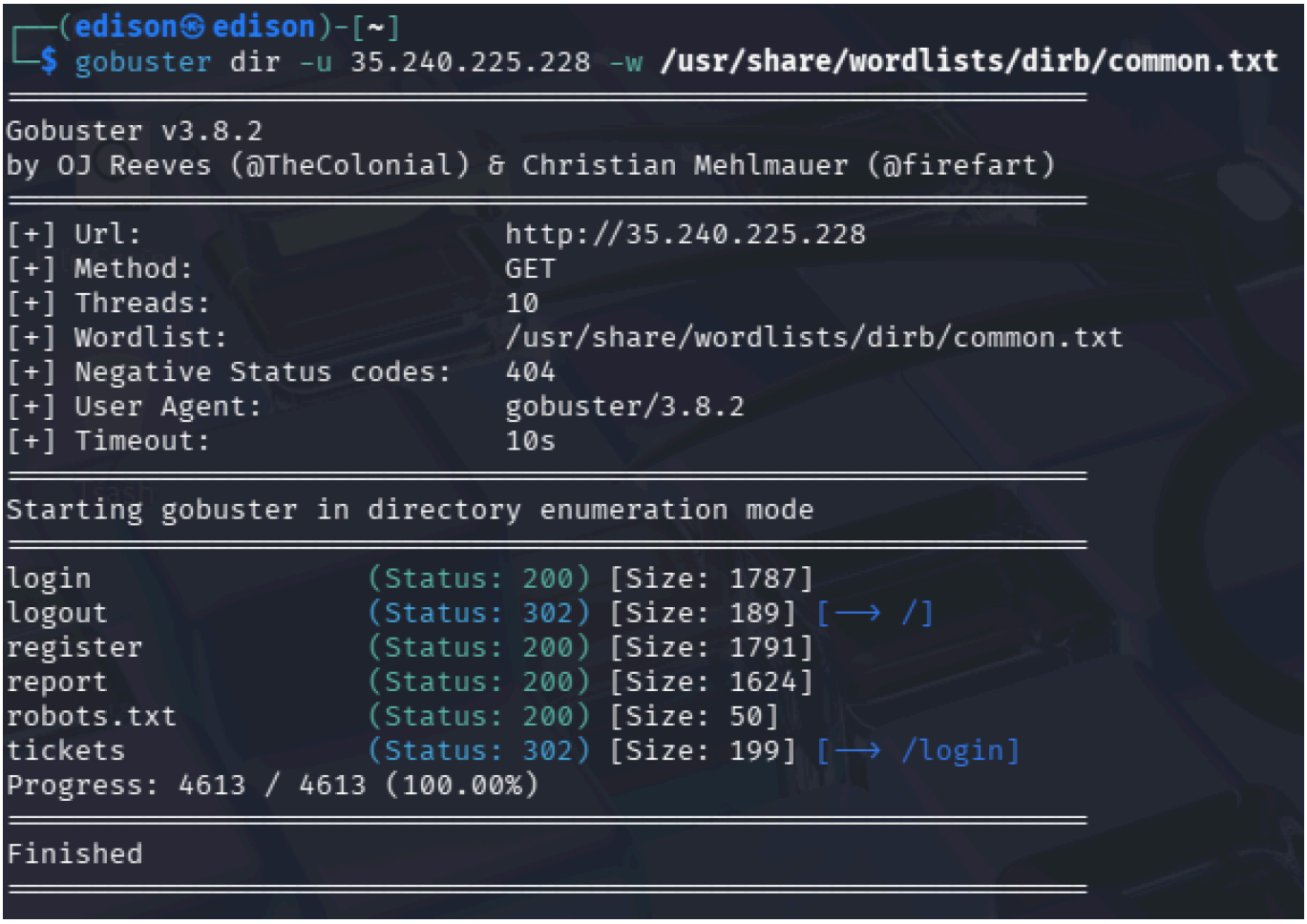


Next, I checked for hidden directories by using the robots.txt file. This revealed the hidden endpoint /admin and /report.

I tried going to the /report endpoint and found a form where I could submit a URL path to my ticket. The page shows that an admin would visit the submitted page to investigate. This make me think an automated bot with admin privileges may be active.

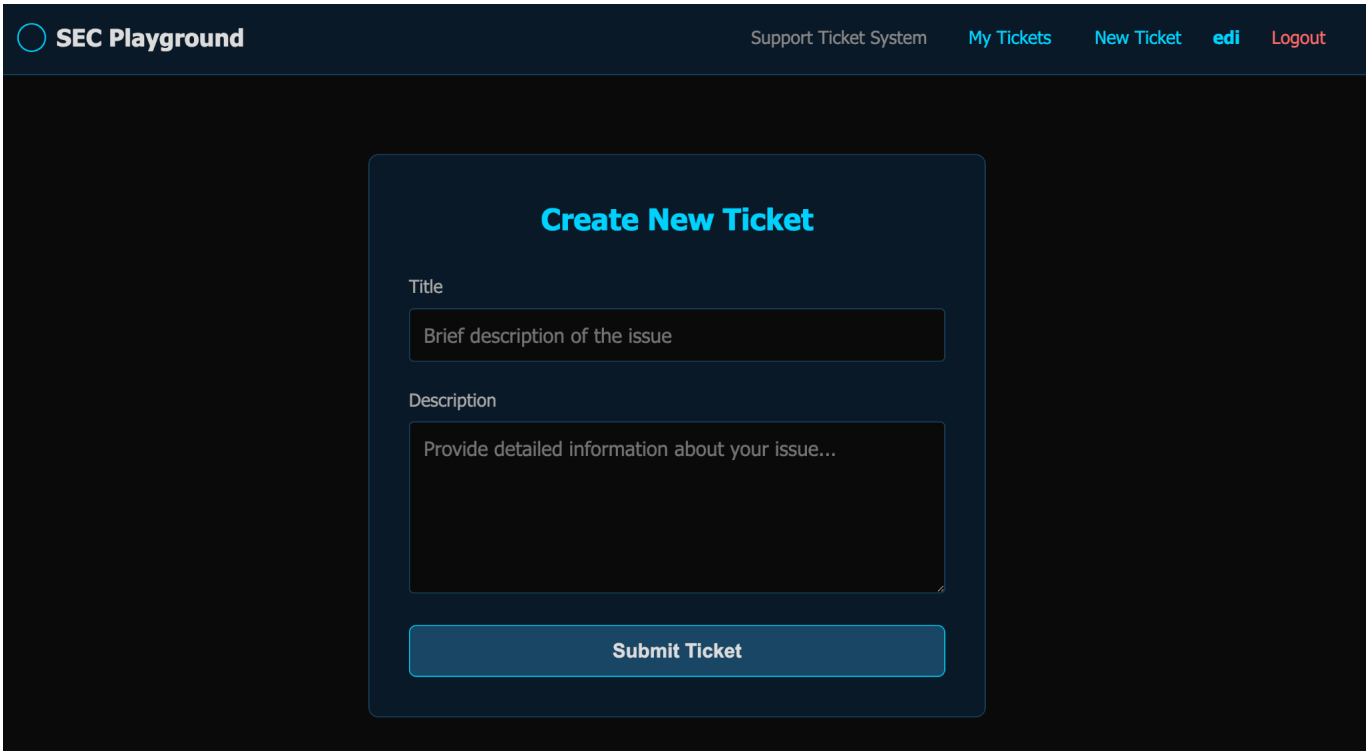


To make sure I don't miss anything, I also ran an automated gobuster directory enumeration scan, which confirmed the existence of the /report, /admin, and /tickets endpoints.

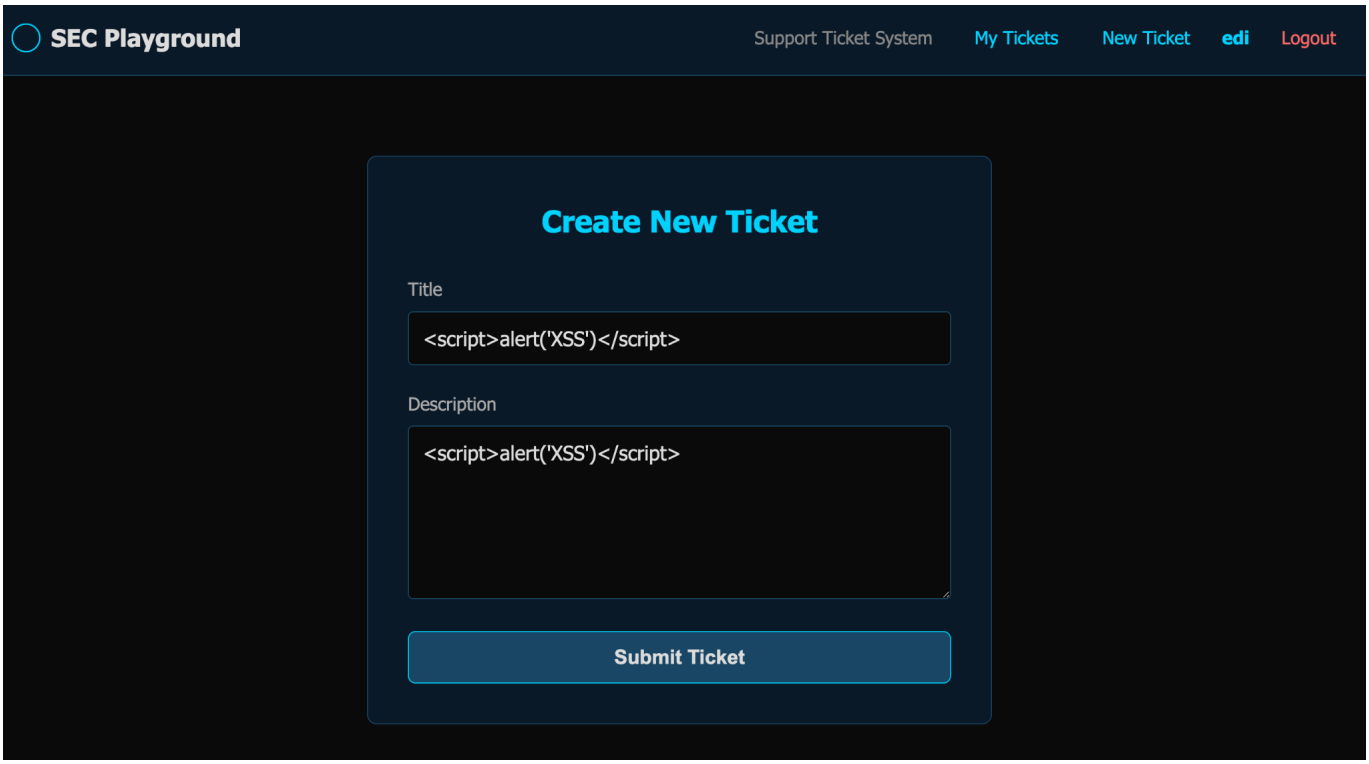


2.2. Vulnerability: Stored XSS

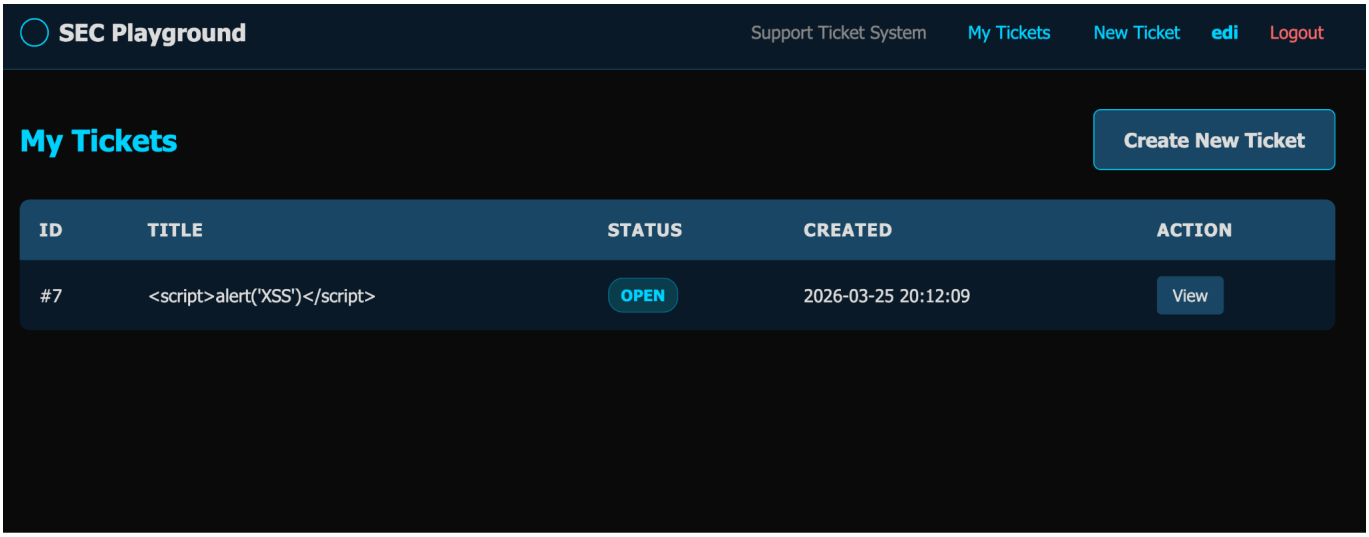
I navigated to the "Create New Ticket" form to test for potential injection flaws.

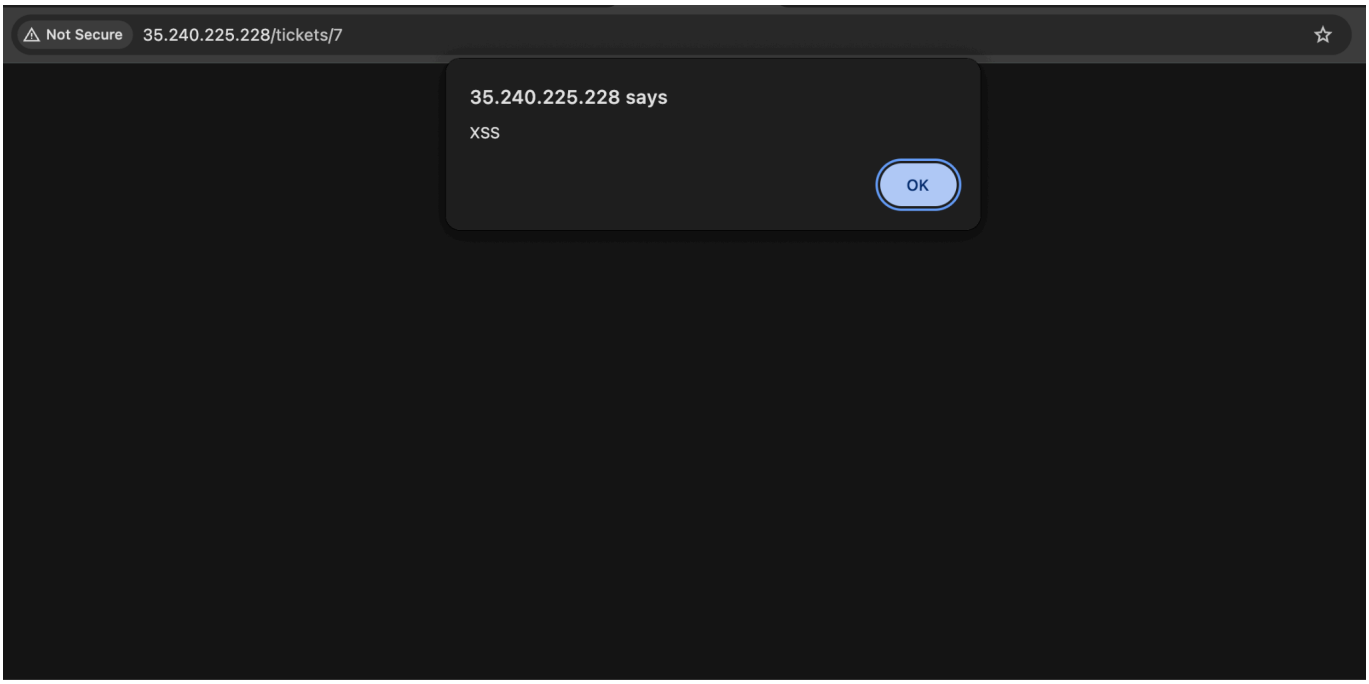


I injected a basic JavaScript payload (<script>alert('XSS')</script>) into both the Title and Description fields.



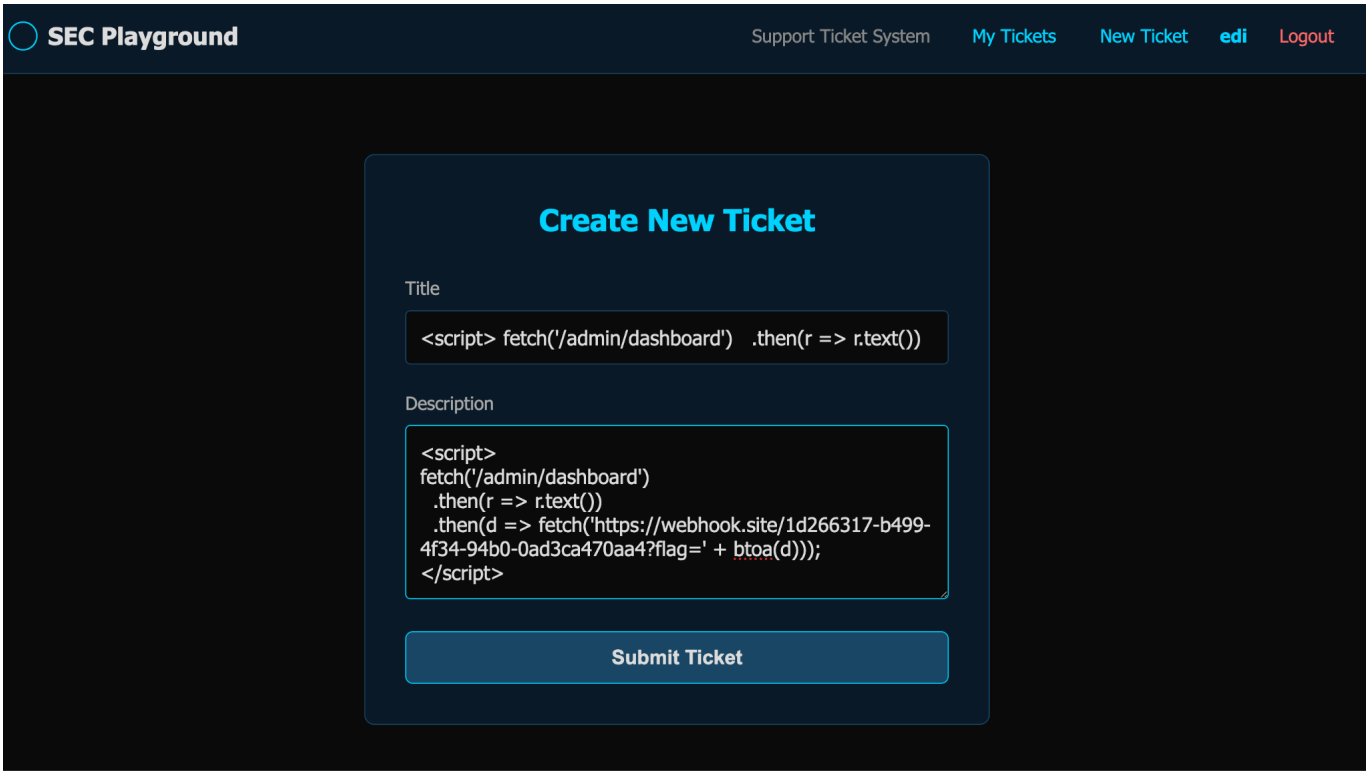
When I opened the newly created ticket, my browser executed the script and displayed an alert box. This confirmed XSS vulnerability because the application was executing my input instead of showing it as text.



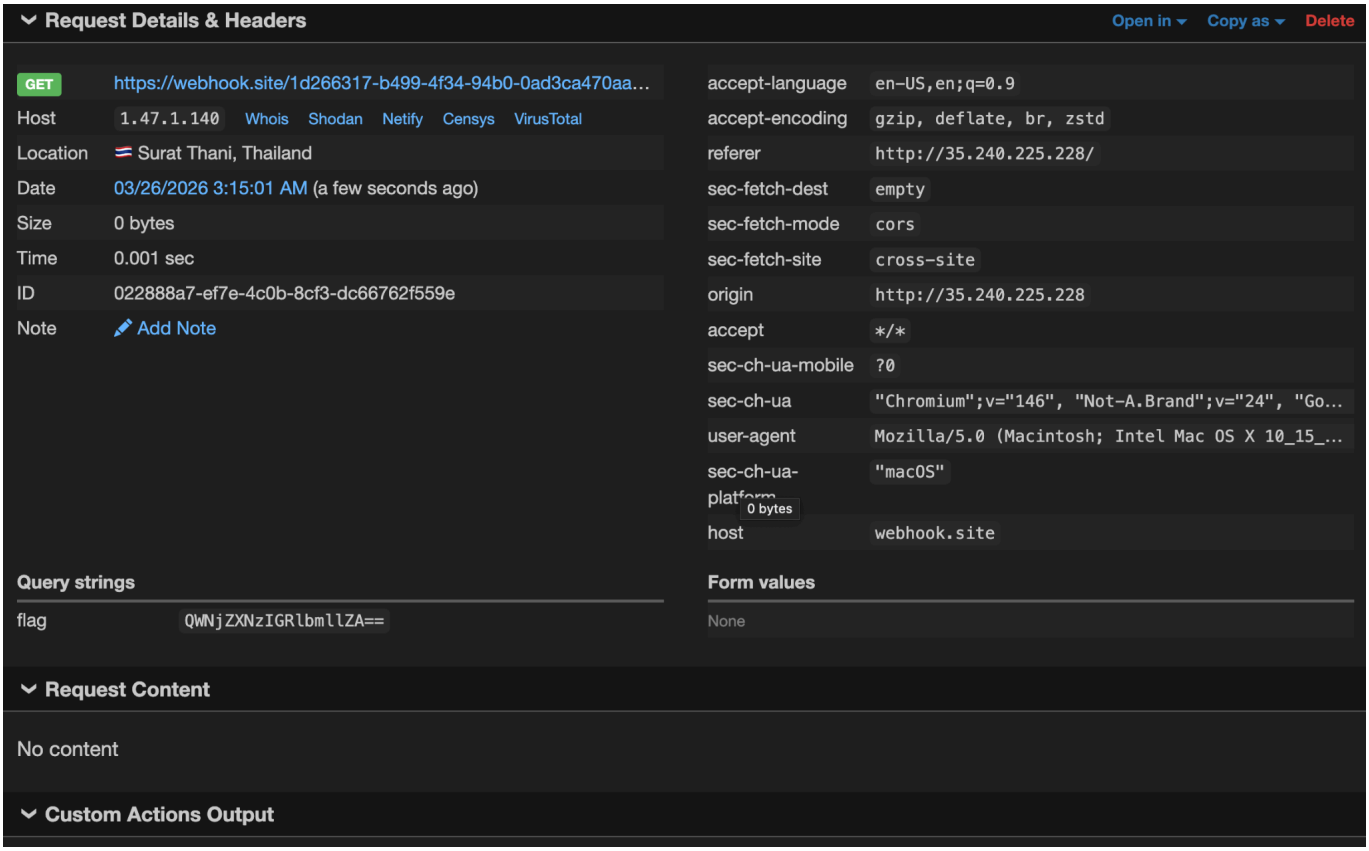


2.3. Exploit & Payload

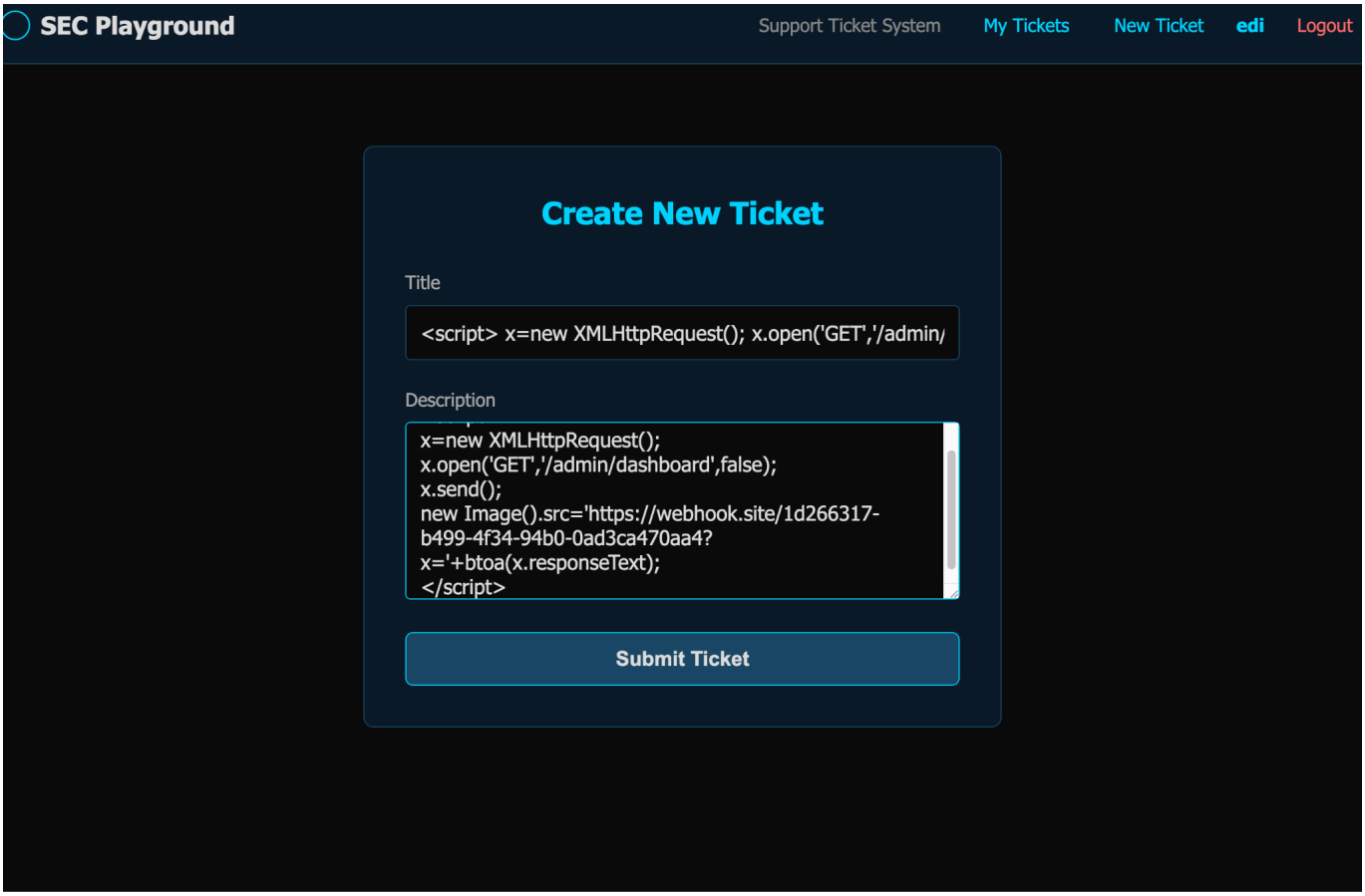
Knowing that an administrator reviews reported URLs, I decided to use the XSS flaw to find the flag. My first attempt was to use the fetch() API to grab the contents of the /admin/dashboard and extract it to my webhook.

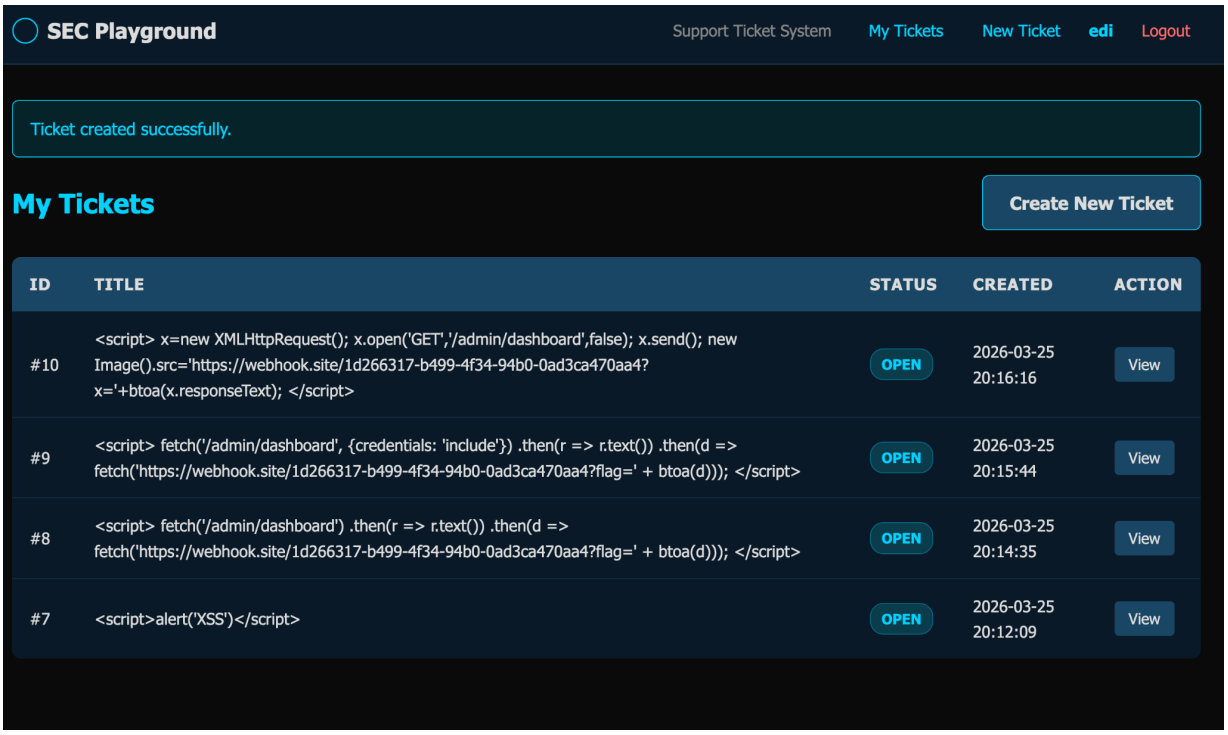


However, when I checked my webhook server, I only received a base64 encoded string that translated to "Access denied".

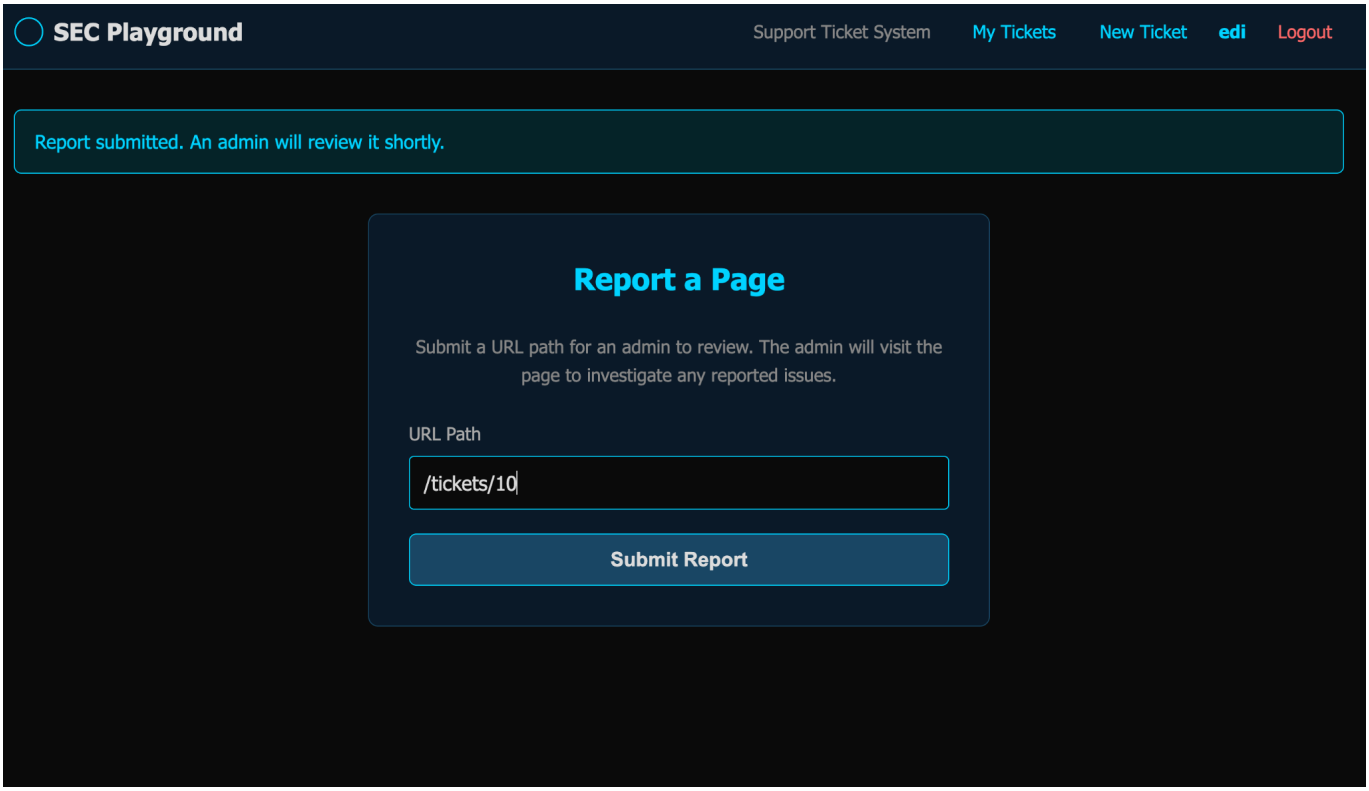


I needed a better approach. I adjusted my payload to a synchronous XMLHttpRequest and an Image source tag for the extraction. I created Ticket #10 with this new payload.



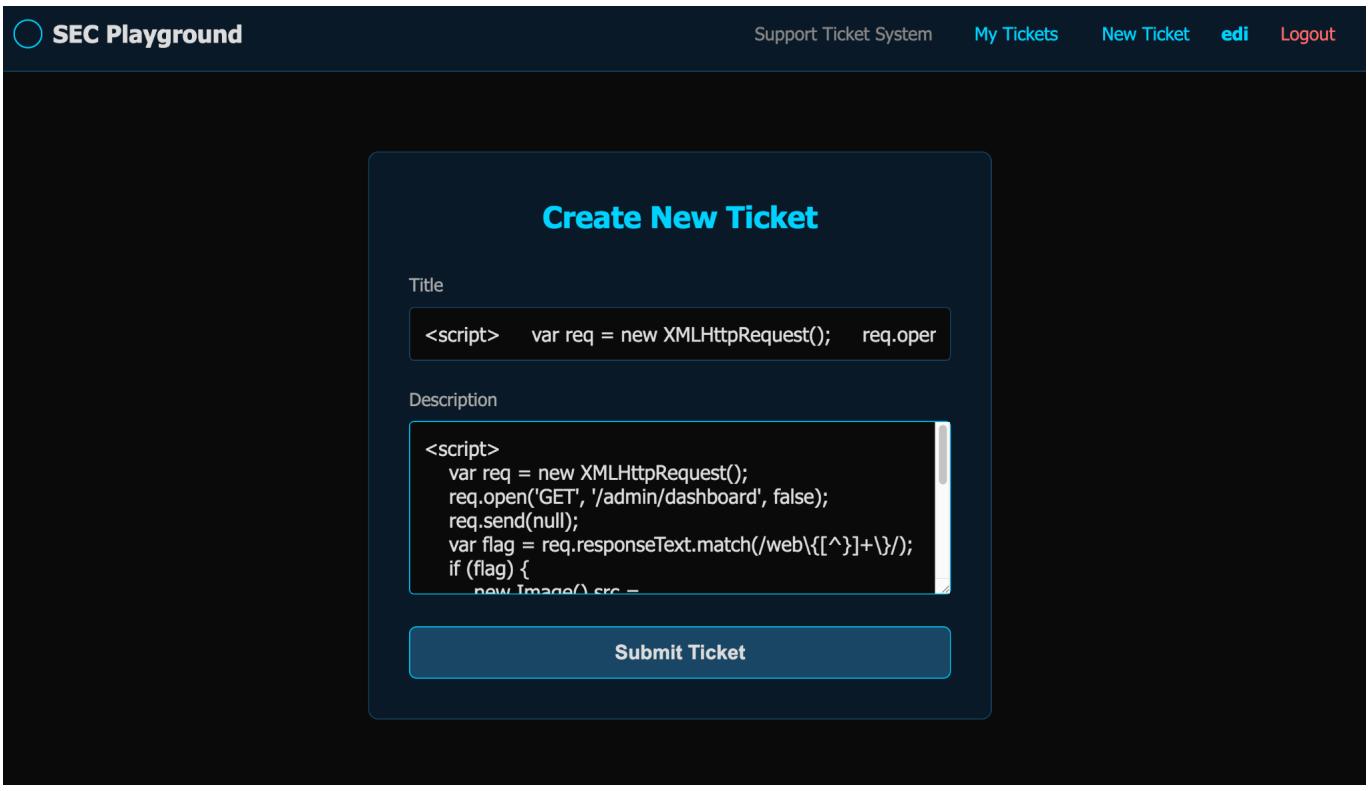


I then submitted the URL path /tickets/10 to the admin bot via the /report endpoint.

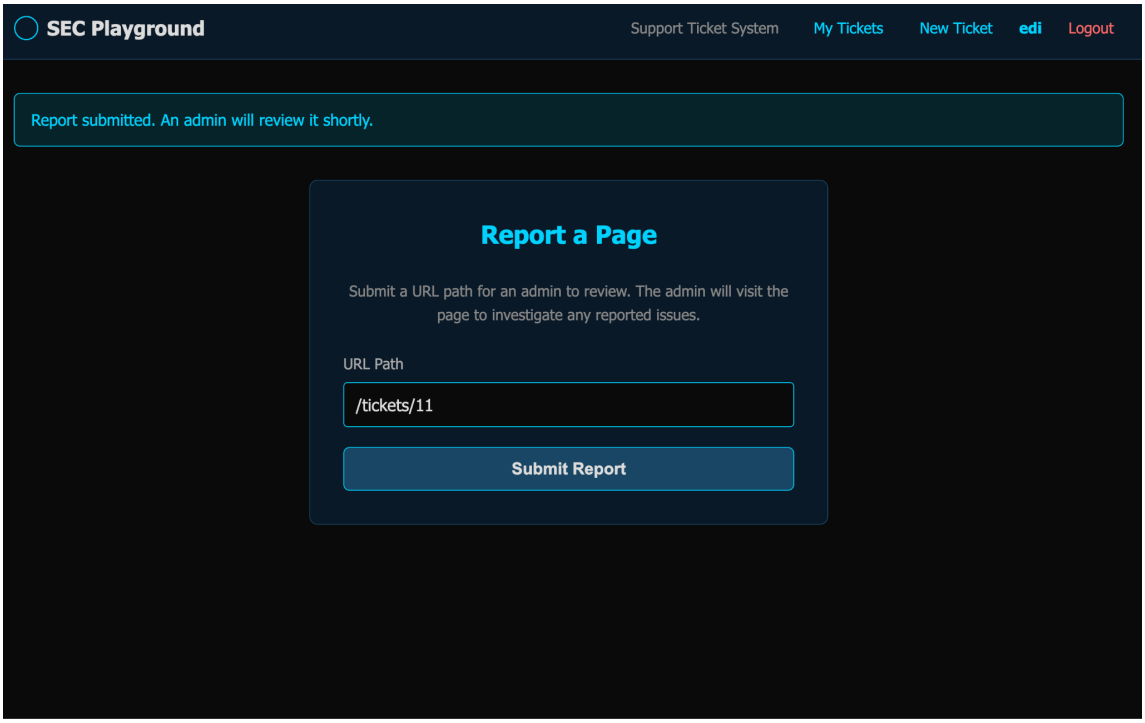


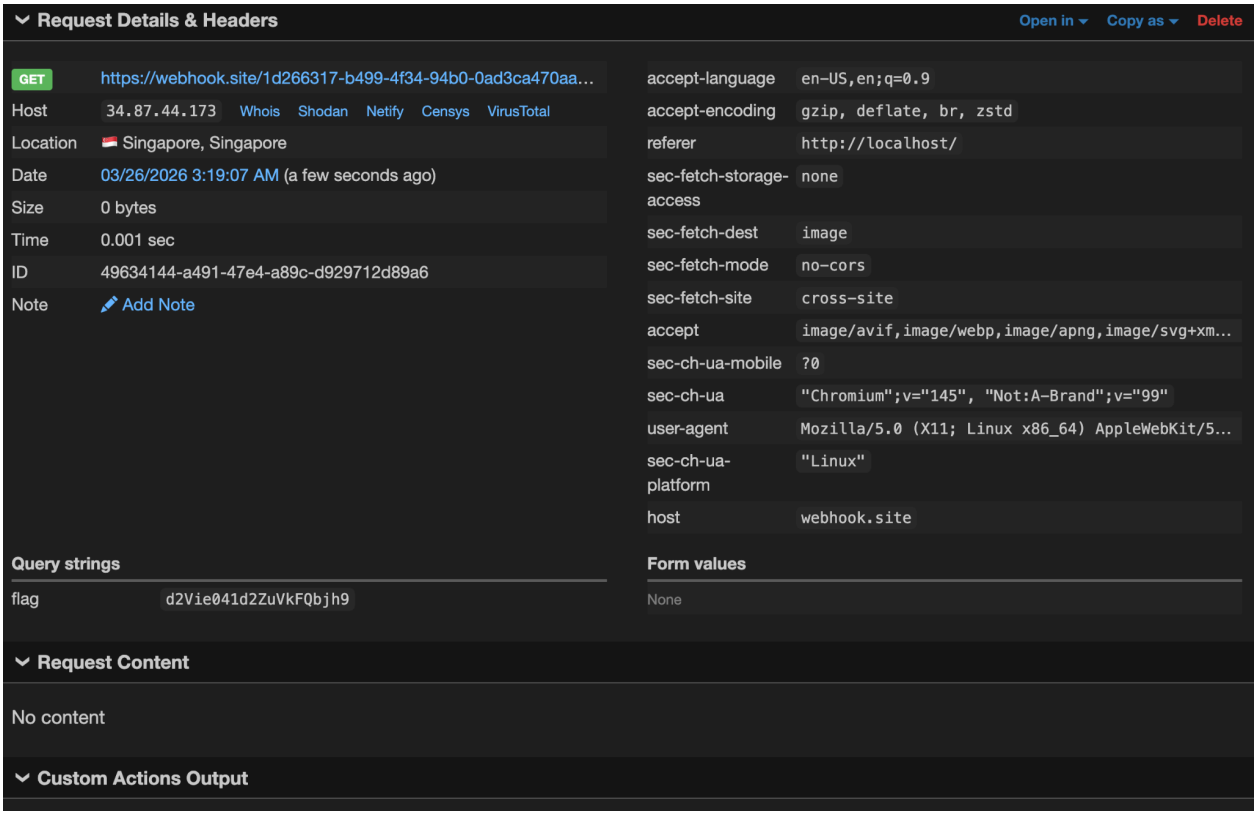
2.4. Successful Flag Extraction

To ensure my payload wouldn't fail due to URL length limits when stealing the entire dashboard HTML, I crafted a better payload (Ticket #11). I used a regular expression (match(/web\{[^\}]+\}/)) to specifically target and extract only the flag string from the admin dashboard.



I submitted Ticket #11 to the admin review bot.





The payload is executed in the administrator's browser. It extracted the flag and sent it to my external server. My webhook captured the base64-encoded query string, which decoded to the final flag: web{N5wfnVAPn8}.

3. Severity / Impact

Overall Risk Level: HIGH (CVSS Score: 8.6)

- **Administrative Abuse:** Because my script ran directly within the admin's browser, I could perform any action on behalf of the administrator. A real attacker could use this to create admin accounts, delete user data, or modify system settings.
- **Data Breach:** I successfully bypassed the server's access controls by forcing the admin's own browser to read restricted data (/admin/dashboard) and hand it over to me.
- **Persistent Threat:** My payload was permanently saved in the application's database. Anyone who views that corrupted ticket will be hacked without having to click on any suspicious links.

4. Remediation

To improve the system in term of security, the following should be applied:

1. **Data Sanitization:** The root cause of this flaw is that the application trusts user input and renders it directly as HTML. All user-supplied data (like Titles and Descriptions) must be strictly encoded before being displayed. Characters such as <, >, &, ', and " must be converted into their safe HTML entities (e.g., <, >).
2. **Add a Content Security Policy (CSP):** Deploy a strict Content Security Policy via HTTP headers. A strong CSP can completely disable the execution of inline scripts (unsafe-inline) and restrict where external data can be sent, which would have neutralized my payloads even after they were injected.
3. **Enforce Strict Session Management:** Ensure that sensitive administrative endpoints require secondary authentication (like password re-entry) for critical actions. This minimizes the damage an attacker can do if they successfully hijack a session via XSS.